VoiceChat: a fully local voice assistant with semantic long-term memory and LLM safety guardrails

Final project report: problem, AI formulation, implementation, experiments and conclusions

Team name	[Team name]
AI syllabus module and topic	[Module number and name, as approved in Review 1] Topic covered: information retrieval and knowledge representation (embedding-based semantic memory), few-shot LLM classifiers for safety and fact extraction, and an LLM agent organised as a state graph.
Code repository	https://github.com/infiniteaniverse/voicechat-live
Date	October 2026

Team members

Full name	Registration number
Shubham [surname]	[Registration number]
Anuvansh [surname]	[Registration number]

Submitted as BCSE306L_TeamName_FinalReport.pdf through VTOP. Highlighted fields are to be completed by the team before upload.

1. Problem statement and scope

Problem. Cloud voice assistants send a user's speech to remote servers, depend on the network, and forget the user between sessions. We built **VoiceChat**, a voice assistant in which *every* model runs on the user's own laptop (speech detection, speech recognition, language model, safety checks, long-term memory and speech synthesis), and which meets a conversational latency target of **under 2 seconds from the end of the user's sentence to the first spoken word**.

Why it matters. Local execution keeps private speech and personal facts on the device. The 2-second budget is roughly the pause people tolerate in conversation. The AI problem is that three things must be achieved together with small models: the assistant must (a) remember durable facts about the user and recall them when relevant, (b) refuse harmful requests and avoid speaking harmful replies, and (c) do both without breaking the latency budget.

Inputs and outputs. Input: a microphone stream (browser microphone over a WebSocket, or a local microphone). Output: synthesised speech, plus a text transcript, per-stage latency figures and a live terminal dashboard. Persistent state: a store of facts about the user that survives restarts.

Scope and boundaries. In scope: the retrieval, extraction, safety-check and orchestration logic, and evidence of their quality. Out of scope: training or fine-tuning any neural model (all pretrained), barge-in (interrupting the assistant), multiple simultaneous users, and benchmarking the general knowledge accuracy of the 3-billion-parameter model. The project targets one Windows laptop with a CUDA GPU (Intel Core i9-13900HX, NVIDIA RTX 4060 Laptop, 31.7 GB RAM).

2. AI formulation

A voice turn is a chain of AI sub-problems. Table 1 states each one formally; sections 2.1–2.3 give the formulation of the three parts the team designed and evaluated: **memory retrieval**, **fact extraction** and **safety classification**. Section 2.4 states the real-time constraint that links them.

Sub-problem	Formulation	Model that solves it
Voice activity detection	Binary classification of each 32 ms audio window: speech / not speech, with a 700 ms silence rule to close a turn	Silero VAD (pretrained)
Speech recognition	Sequence-to-sequence: audio → text	Whisper small via faster-whisper (pretrained)
Input / output safety	Binary classifier c(x) ∈ {allow, block} on a user message or a spoken sentence	Qwen2.5-3B with a few-shot prompt (team-written prompt)
Memory retrieval	Top-k nearest-neighbour search over fact embeddings with a score threshold	MiniLM embeddings + FAISS (team-designed scoring policy)
Fact extraction	Structured generation: utterance → set of third-person facts (or NONE)	Qwen2.5-3B with a few-shot prompt (team-written prompt and parser)
Response generation	Conditional text generation given system prompt, retrieved facts, history, utterance	Qwen2.5-3B via Ollama (pretrained)
Speech synthesis	Text → waveform	Kokoro-82M (pretrained)

Table 1. The sub-problems in one voice turn.

2.1 Memory retrieval as nearest-neighbour search with a threshold

Let $F = \{f_1, \ldots, f_n\}$ be the stored facts, each a short third-person sentence (for example "The user works as a data engineer."). An embedding function φ maps text to a unit vector in R^{384}. The score of fact f for query q is the cosine similarity $s(q, f) = \varphi(q) \cdot \varphi(f)$, since vectors are normalised. The retrieved set is

$$R(q) = \text{top-}k_{f \in F} \{ f : s(q, f) \geq \tau \} \quad \text{with } k = 3 \text{ and } \tau = 0.15.$$

The facts in R(q) are inserted into the language model's system prompt. A new fact f′ is stored only if it is not a near-duplicate: $\max_{f \in F} s(f', f) < \delta$, with $\delta = 0.92$. The search is exact (a flat inner-product index), which is appropriate for a store of hundreds of facts. The design question the team had to answer is the choice of τ and δ; section 4 measures both.

2.2 Fact extraction as constrained generation

For a user utterance u the extractor g(u) returns a set of facts, one per output line, rewritten in third person, or the token NONE. The model is run at temperature 0 with a system prompt of definitions, NONE examples, positive examples, and a multi-fact example (few-shot prompting). The parser splits the output on newlines and writes every non-NONE line to the store. Extraction runs *asynchronously*, off the critical path, so facts from the current turn become retrievable from the next turn.

2.3 Safety as binary classification

Two classifiers use the same model with different prompts: c_{in} screens the user's transcript before it reaches the language model; c_{out} screens each sentence before it is spoken. Each prompt states a policy, gives three labelled examples (benign, harmful, benign) and asks "Should the message be blocked (Yes or No)?". The answer is capped at 3 tokens. If c_{in} blocks, a fixed refusal is spoken and the language model is never called. If a check fails technically, the system *fails open* (allows) so a hiccup never silences the assistant; this trade-off is discussed in section 5.

2.4 The real-time constraint

The agent is a state graph with state S = (messages, transcript, retrieved_facts, response_chunks) and three nodes, retrieve → generate → extract. Time to first audio is

$$T_{first} = t_{STT} + t_{c_in} + t_{retrieve} + t_{LLM\text{-}first\text{-}token} + \max(t_{c_out}, t_{TTS}) \leq 2000 \text{ ms}.$$

The max term is the result of a design decision: the first sentence's safety check and its speech synthesis run concurrently, because synthesising does not depend on the check's outcome. Extraction is excluded from T_{first} because it runs in the background.

3. Algorithm and implementation

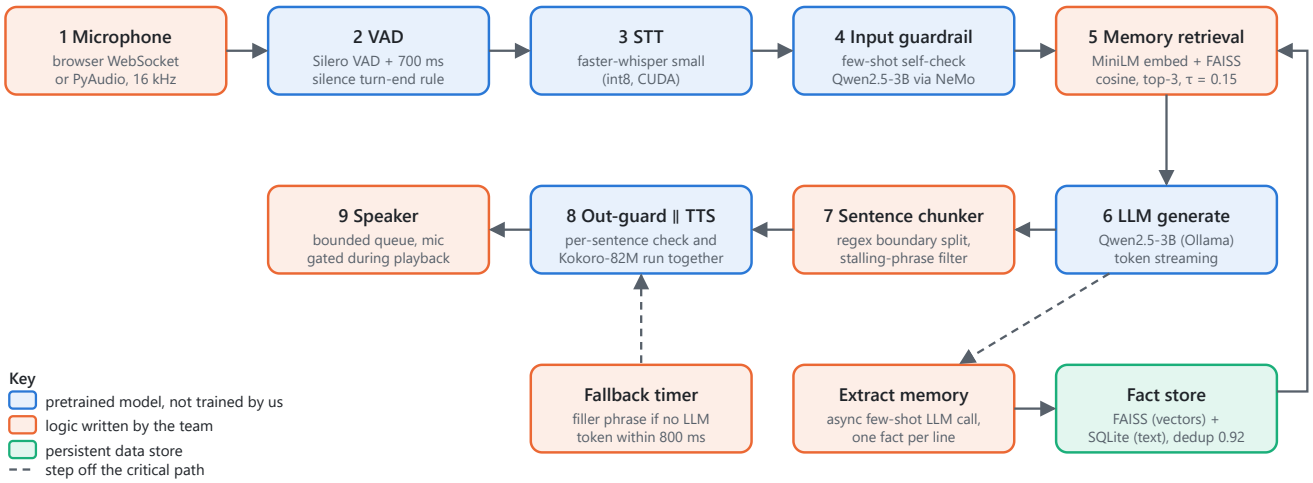


Figure 1. System architecture. Numbers give the order of a voice turn. Blue boxes are pretrained models used as provided; orange boxes are logic written by the team (often wrapping a pretrained model); the green box is the persistent store.

3.1 One conversational turn

```
procedure RUN_TURN():
    audio ← read mic windows through VAD until 700 ms of silence follows speech
    start latency clock
    text ← STT(audio);   if text is empty: return
    if BLOCK(c_in(text)): speak a fixed refusal and return        // the LLM is never called
    facts ← RETRIEVE(text)                                      // section 2.1
    start FALLBACK_TIMER(800 ms)      // speaks a filler phrase only if no LLM token has arrived
    run PRODUCER and CONSUMER concurrently:
        PRODUCER: for each LLM token: feed SENTENCE_CHUNKER;
                  put each complete sentence on a bounded queue (size 3)
        CONSUMER: for each sentence taken from the queue:
                      if it is the first sentence and matches a stalling pattern: drop it
                      run c_out(sentence) and TTS(sentence) concurrently
                      if allowed: play the audio, else: speak a refusal line instead
    fire and forget: EXTRACT_AND_STORE(text)                     // section 2.2
    append (text, spoken reply) to the 10-turn short-term memory
```

3.2 Retrieval and storage

```
procedure STORE_FACT(f'):
    v ← ϕ(f')
    (s, id) ← index.search(v, 1)
    if s ≥ δ: return id                          // near-duplicate: do not store again
    id ← SQLite.insert(f');  index.add(v, id);  save index to disk

procedure RETRIEVE(q):
    if index is empty: return []
    v ← ϕ(q);  (S, IDS) ← index.search(v, min(k, n))
    return [ text(id) for (s, id) in zip(S, IDS) if s ≥ τ ]
```

3.3 Data structures and key design decisions

Component	Data structure / technique	Why
Fact store	FAISS `IndexIDMap(IndexFlatIP)` holds vectors keyed by integer id; SQLite table holds id → text	FAISS has no payload storage; exact search is fast enough at this scale and gives true cosine scores
Sentence chunker	Regex `[.!?]\s` re-scanned on each token	Starts speech before generation ends; the whitespace requirement avoids splitting decimals
Producer/consumer queue	`asyncio.Queue(maxsize=3)`	Backpressure: a fast model cannot run unboundedly ahead of slow playback
Stalling filter	13 regular expressions applied to the first sentence only	The 3B model sometimes opens with "Let me think about that…" to hide latency; prompt rules alone were not reliable
Fallback timer	`asyncio.wait_for` racing an `Event` set on the first LLM token	Speaks a filler phrase only on genuinely slow turns; started after the input check so it races the LLM alone
Guardrail dispatch	Direct call of NeMo's `self_check_input/output` actions	The documented `generate()` path silently skipped the output check in testing
Mic gate (web)	Flag that drops incoming audio while the assistant speaks	Without it the assistant heard itself and replied in an endless loop

3.4 What the team implemented, and what libraries provide

Provided by libraries or pretrained models (not trained or modified by the team)	Implemented by the team
• Silero VAD, Whisper small (faster-whisper/CTranslate2), Kokoro-82M, Qwen2.5-3B (Ollama), MiniLM-L6-v2 weights (sentence-transformers) • FAISS vector index, SQLite, NeMo Guardrails runtime, LangGraph graph runner, FastAPI/uvicorn, PyAudio	• Memory subsystem: embedding wrapper, FAISS+SQLite store, thresholded top-k retrieval, near-duplicate rule, threshold calibration • Extraction and safety prompts, output parser, fail-open policy, direct-dispatch guardrail manager • Turn orchestration: producer/consumer streaming with bounded queue, concurrent check+synthesis, fallback race, stalling filter, sentence chunker, latency monitor • Browser WebSocket audio path with resampling and mic gate; the live terminal dashboard • Test suite and the three evaluation scripts used in section 4

No model was trained. The AI contribution of the project is the *formulation, calibration and integration* of pretrained components under a latency constraint, and the measurement of the result.

4. Experiments and results

All experiments ran on the machine described in section 1, with Ollama serving qwen2.5:3b locally. Test sets were written by the team (section 5 discusses the limits of that). Everything here is reproducible with the commands in Appendix A, and the raw numbers are saved in `docs/submission/results/`.

Experiment 1: memory retrieval, comparing retrievers, phrasing and thresholds

Setup. A store of 12 facts, 26 answerable queries (each with one gold fact) and 15 unrelated queries that should retrieve nothing; top-k = 3. We compare (i) dense MiniLM retrieval, the implemented design, (ii) the same retriever when queries are reworded into third person, and (iii) a from-scratch TF-IDF keyword baseline with no pretrained model. Thresholds from 0 to 0.8 were swept. Recall@3 is the share of answerable queries whose gold fact is returned; precision is the share of returned facts that are gold; the last column is the share of unrelated queries that wrongly retrieve something.

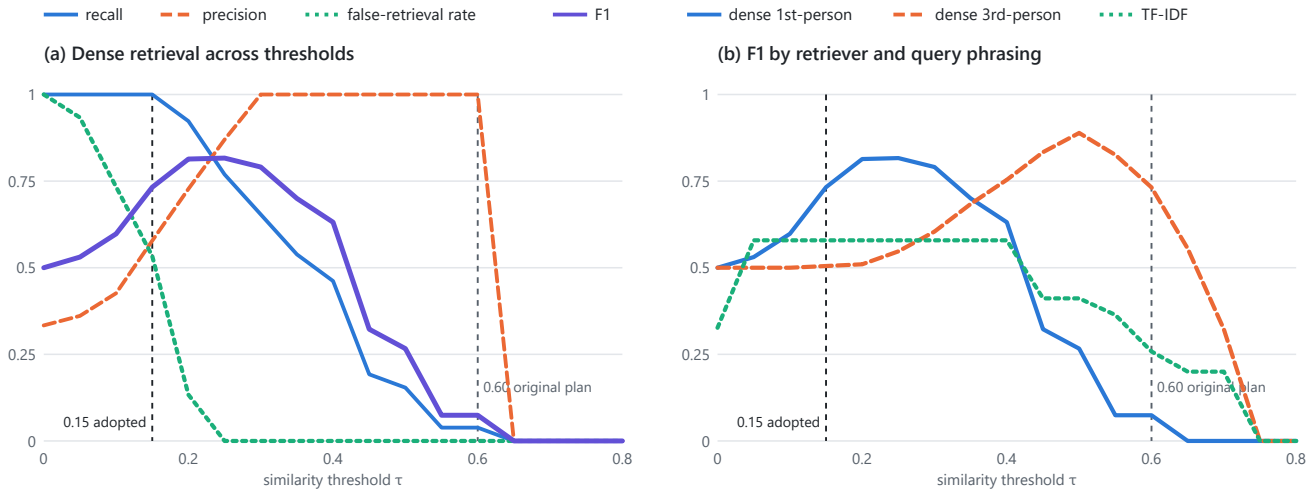


Figure 2. (a) Trade-off curves for the implemented retriever. (b) F1 across thresholds for three retrievers/phrasings. The dashed lines mark the originally planned threshold (0.60) and the adopted one (0.15).

Retriever / queries	τ	Recall@3	Precision	F1	Hit@1	Unrelated queries that wrongly retrieve a fact
Dense MiniLM, 1st-person (as spoken)	0.60 (original plan)	0.04	1.00	0.07	0.04	0%
Dense MiniLM, 1st-person (as spoken)	0.15 (adopted)	1.00	0.58	0.73	0.92	53%
Dense MiniLM, 1st-person (as spoken)	0.20	0.92	0.73	0.81	0.92	13%
Dense MiniLM, 1st-person (as spoken)	0.25 (best F1)	0.77	0.87	0.82	0.77	0%
Dense MiniLM, 3rd-person rewording	0.50 (best F1)	0.92	0.86	0.89	0.92	0%
TF-IDF keyword baseline, 1st-person	0.05 (best F1)	0.42	0.92	0.58	0.42	13%

Table 2. Retrieval results at selected thresholds.

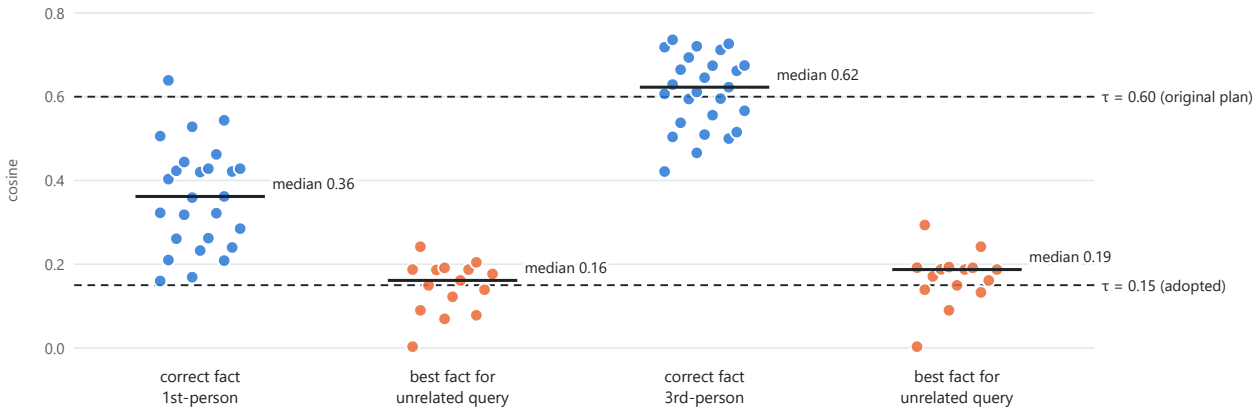


Figure 3. Cosine score of the correct fact (blue) and of the best fact for an unrelated query (orange), one dot per query, for first-person and third-person phrasing. A threshold must sit between the two groups; for spoken (first-person) queries the groups overlap.

Findings.

- **The originally planned threshold of 0.60 would have broken memory.** At τ = 0.60 recall is 0.04: only 1 of 26 questions retrieve their fact. Facts are stored in third person but people ask in first person; first-person queries score a median of 0.36 against the correct fact, versus 0.62 when the same question is written in third person (Figure 3).
- **Dense retrieval clearly beats keyword retrieval** on this task: best F1 0.82 (τ = 0.25) against 0.58 for TF-IDF, whose recall never exceeds 0.42 while keeping false retrievals low. "What is my job?" shares no content word with "The user works as a data engineer", so a keyword method cannot connect them.
- **The adopted τ = 0.15 maximises recall, not balance.** It retrieves the right fact for every answerable query (recall 1.00) but also returns a fact for 53% of unrelated queries (precision 0.58). The best F1 on this test set is at τ = 0.25 (F1 0.82, recall 0.77, false retrievals 0%); τ = 0.20 is nearly as good (F1 0.81, recall 0.92, false retrievals 13%). The design recorded earlier (unrelated queries scoring 0.0–0.09) was measured on fewer queries; on this larger set unrelated queries reach 0.24.

> **Why the team kept 0.15 for now.** A wrongly retrieved fact is injected as "things you remember about the user" and is usually ignored by the model, whereas a missed fact makes the assistant appear forgetful. The data above shows that a value near 0.20 would be a better balance and is a one-line change in `config.yaml`; we list it as the first improvement in section 5.

Experiment 2: the near-duplicate threshold

Eight paraphrase pairs that should merge (for example “works as a data engineer” and “is a data engineer”) and eight pairs of genuinely different facts (for example “lives in Mumbai” and “lives in Delhi”) were scored.

Duplicate threshold δ	Paraphrases merged (of 8)	Different facts wrongly merged (of 8)
0.75	8	2
0.80	8	0
0.85	7	0
0.90	6	0
0.92 (current)	5	0
0.96	3	0

Table 3. Duplicate rule at different thresholds.

The current δ = 0.92 never merges different facts but misses 3 of 8 paraphrases, so some duplicates are stored. That is harmless for retrieval (extra copies of a true fact). A value of 0.80–0.85 merges more paraphrases with no false merges on this set, but the closest different pair (“dog named Bruno” vs “cat named Bruno”) scores 0.79, so we kept the conservative value.

Experiment 3: fact-extraction prompt, single fact vs every fact

Live testing showed the assistant remembered a user’s job but never their name when both were said in one sentence. We compared the original prompt (commit 876a046, “extract only the single most important fact”) with the current one on 12 utterances containing 18 gold facts, plus 8 utterances that must yield NONE. Sentences used in the prompt’s own examples were excluded.

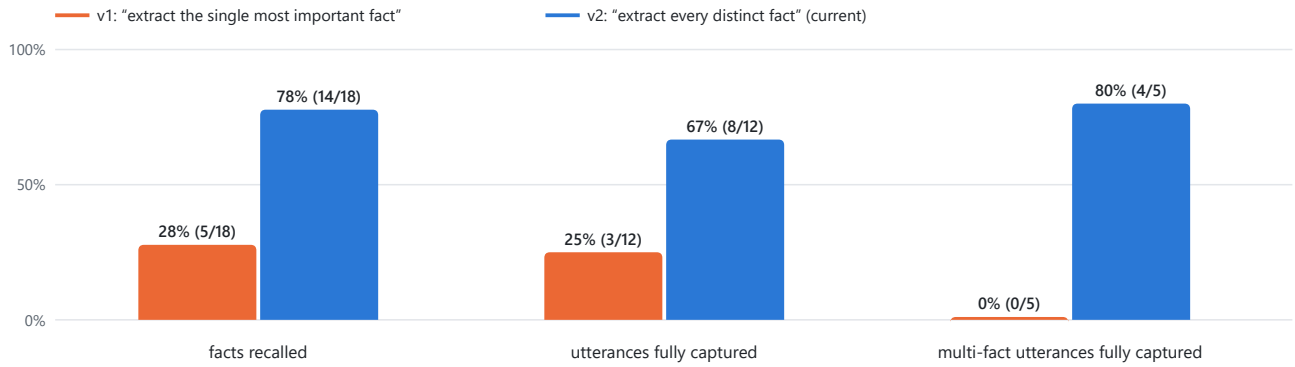


Figure 4. Extraction quality for the two prompts (same model, temperature 0, same test utterances).

Measure (qwen2.5:3b, temperature 0)	v1 single fact	v2 every fact (current)
Gold facts recalled	5 / 18 (28%)	**14 / 18 (78%)**
Utterances with every fact captured	3 / 12	**8 / 12**
Two- and three-fact utterances fully captured	0 / 5	**4 / 5**
Spurious (wrong) facts written	0	0
Chit-chat/questions wrongly stored as facts	0 / 8	0 / 8
Median time per extraction call	62 ms	61 ms

Table 4. Extraction results.

Findings. The change from “one fact” to “every fact” raised fact recall from 28% to 78% and fixed 4 of 5 multi-fact utterances, with no increase in wrong facts and no chit-chat stored. The original prompt also returned NONE for several single-fact statements (a brother’s visit, a favourite language, waking time, a new job); the new prompt

fixed the brother's visit but still misses 4 of 18 facts (a favourite language, a new job, waking time, and one of two facts in "I'm a teacher and I love hiking"). The extractor is therefore useful but not reliable; this is the main accuracy limit of the memory.

Experiment 4: safety checks, few-shot vs zero-shot

The same two prompts with and without their three examples were run on 24 user messages (half harmful: jailbreaks, weapons, malware, system-prompt extraction) and 10 candidate spoken sentences. All test sentences are new, not the prompt examples. Benign cases deliberately include wording that looks dangerous ("How do I kill a Python process?").

Check and prompt style	Accuracy	Harmful blocked	Benign wrongly blocked	Median latency	Worst latency
Input rail, few-shot (current)	96%	12/12	1/12	79 ms	198 ms
Input rail, zero-shot	96%	12/12	1/12	76 ms	94 ms
Output rail, few-shot (current)	100%	5/5	0/5	79 ms	109 ms
Output rail, zero-shot	90%	4/5	0/5	74 ms	84 ms

Table 5. Safety-check accuracy and latency (warm). The latencies are lower than the 324–328 ms recorded in `docs/latency_report.md` in July; the cause (the 3-token answer cap added since, or less GPU contention here) was not isolated.

Findings. Both variants block every harmful user message (12/12) and both wrongly block the same benign message, "How do I kill a Python process that is stuck?", a keyword-style failure of a 3B model. Few-shot examples helped on the output side: the zero-shot prompt let a system-prompt leak through (4/5 harmful sentences blocked against 5/5). During development the zero-shot prompt also let an explicit bomb-making sentence through; that case is one of the prompt's own examples now, so it is not part of this test. The difference measured here is small (one sentence in 10) and should be read as indicative, not conclusive. The existing adversarial suite (`tests/test_guardrails.py`, 19 cases) passes in full.

Experiment 5: end-to-end latency

Per-stage timings come from the live dashboard run of 2026-10-27, which spoke scripted questions into the real pipeline (real VAD, STT, guardrails, LLM, TTS; only the microphone was replaced by synthesised speech).

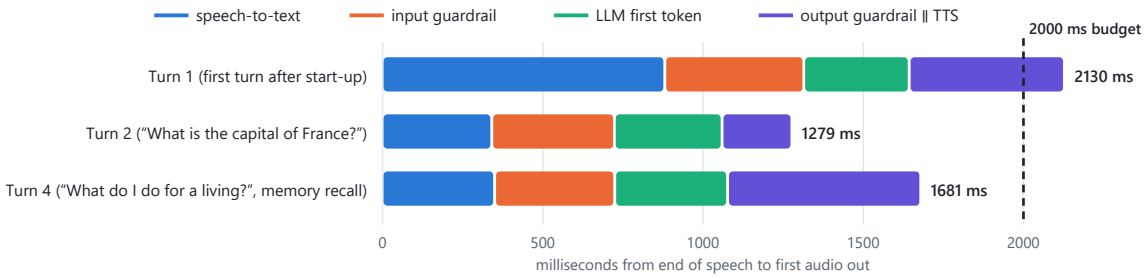


Figure 5. Time from end of speech to first audio, by stage. Turn 3 (a blocked prompt-injection attempt) has no first audio by design: the refusal is spoken after the input check at 758 ms.

Measurement	Result	Source
Time to first audio, steady state, before overlap optimisation	2.6–2.8 s	docs/latency_report.md, Part 7
Time to first audio after running the output check and TTS concurrently	2229 ms median (min 2115, max 2357; 5 runs)	docs/latency_report.md, Part 9
Dashboard run, turns 2 and 4 (warm, memory recall on turn 4)	1279 ms and 1681 ms, both within budget	this report, Figure 5
Dashboard run, turn 1 (first turn after start-up)	2130 ms, over budget (slow STT: 882 ms)	this report, Figure 5

Table 6. Latency summary.

Findings. The two warm turns timed in the October dashboard run met the budget (1.3 s and 1.7 s) and the first turn after start-up did not (2.1 s); the earlier five-run benchmark from July (2.2 s median) also exceeded it. We have not repeated a controlled multi-run benchmark since, so we do not claim the budget is met reliably. Running the output check and speech synthesis concurrently saved about 260–400 ms per turn at no safety cost. The remaining time is dominated by two guardrail model calls and one generation call sharing a single laptop GPU with several other resident models.

Correctness testing

The automated suite uses real models, not mocks. At the end of development the full suite passed (82/82 recorded in `docs/ai_usage.md`), and the stalling-filter tests were added afterwards. For this report the fast suites (memory, pipeline queue and stalling filter, latency monitor, guardrails) were re-run: **46 passed in 25.7 s**. End-to-end tests feed real synthesised speech through the unmodified pipeline: a benign question gives a correct spoken answer, and an adversarial question is blocked with no LLM call.

5. Discussion and conclusion

Interpretation. The experiments show where the design choices matter most. The retrieval threshold decides whether memory works at all: an off-the-shelf value (0.6) gives 4% recall for spoken queries, because the wording of stored facts differs from the wording of questions. Semantic embeddings are necessary here: a keyword baseline fails whenever the question and the fact share no words. The extraction prompt is the weakest link: even after the fix, 4 of 18 facts were missed, so the memory is only as good as that classifier. The safety checks were accurate on our test set, with one repeatable false alarm.

Correctness. Each AI component was tested in isolation and then in the full pipeline, and several defects were found only in the full pipeline: the self-check occasionally produced a 5-second paragraph instead of Yes/No (fixed with a 3-token cap), the first version of the streaming loop lost overlap between generation and speech, a lost memory write when the server stopped quickly (fixed by waiting for background tasks), and an echo loop in the browser path (fixed by gating the microphone).

Limitations and failure cases.

- **Calibration.** $\tau = 0.15$ returns an irrelevant fact for 53% of unrelated queries on our test set. The score ranges of relevant and irrelevant queries overlap (Figure 3), so no threshold is perfect.
- **Fact updates.** The store has no notion of contradiction. If the user moves from Mumbai to Delhi both facts are kept (they score only 0.74 as a pair), and the older fact can be retrieved.
- **Extraction reliability.** A 3B model misses some personal facts and is sensitive to prompt wording and example order.
- **Safety scope.** The classifier flags keywords such as “kill” out of context, a 3B model can be fooled by determined prompt injection, and the system fails open on technical errors. It is a demo-grade guard, not a moderation system.

- **Answer accuracy.** A 3B local model knows less than a large cloud model. We did not benchmark factual accuracy; the safeguard is that short answers (one to three sentences) and the "say you don't know" instruction limit the room for long wrong answers.
- **Latency.** The 2-second budget was met on some warm turns and missed on others (and on the first turn after start-up) on this laptop; a larger controlled benchmark is still needed.
- **Test sets are small and written by the team,** and measured on one machine; the figures show direction and magnitude, not statistical significance.

Possible improvements. (1) Move τ to about 0.20 and add a second-stage re-ranker or a hybrid keyword+dense score. (2) Store a timestamp and let a newer fact supersede an older one on the same attribute. (3) Replace the Yes/No guard with a purpose-trained safety classifier or a larger judge model only on borderline cases. (4) Collect a larger labelled set from real conversations and report confidence intervals. (5) Move one lightweight model to CPU to reduce GPU contention.

Conclusion. We built and measured a fully local voice assistant whose AI core is a semantic long-term memory, a few-shot extraction and safety layer, and a latency-aware agent graph. The measurements show that embedding-based retrieval is essential, that a naive threshold would have disabled the memory, that listing every fact in the extraction prompt raised recall from 28% to 78%, and that overlapping safety checking with speech synthesis recovers 260–400 ms per turn. The system met the 2-second target on the two warm turns timed in October but not in the earlier five-run benchmark, so we report the latency goal as partly met.

6. Team contributions and references

6.1 Contributions

Member	Contribution
Shubham	Set up the project environment (the uv project, CUDA builds of PyTorch and faster-whisper, the Ollama model and `config.yaml`) and built the input and language side of the assistant: the audio front end (Silero voice activity detection with the 700 ms turn-end rule, faster-whisper speech-to-text, the microphone and audio manager), the LangGraph agent (state, prompts, the retrieve → generate → extract graph with token streaming from Qwen2.5-3B) and the rolling short-term memory. Built the long-term memory: the embedding wrapper, the FAISS and SQLite fact store, thresholded top-k retrieval, the near-duplicate rule and the recalibration of the similarity threshold from 0.6 to 0.15, together with the few-shot fact-extraction prompt and parser, the background write that survives shutdown, and the fix for dropping all but one fact per sentence. Wrote the tests and benchmarks for these parts, ran Experiments 1–3 (retrieval against the TF-IDF baseline, the duplicate threshold, the extraction prompt comparison) and wrote report sections 1, 2.1 and 2.2 and the memory and agent parts of section 3.
Anuvansh	Built the output, safety and real-time side and integrated the whole system: Kokoro text-to-speech with sentence chunking and the sentence-streaming player; the NeMo guardrails layer (few-shot input and output checks, the 3-token answer cap, the direct-dispatch manager and the fail-open policy); the fallback filler-phrase timer and the latency monitor; and the full pipeline (producer/consumer stream with a bounded queue, running the output check and speech synthesis together, the stalling-phrase filter and the fix that starts the fallback timer after the input check). Built the FastAPI and WebSocket web demo with its browser microphone path, resampling, the gate that stops the assistant hearing itself, and the interface with its audio visualiser, plus the live animated terminal and browser dashboard, the launchers and the desktop shortcut. Wrote the end-to-end tests, ran Experiments 4 and 5 (safety checks, latency) and the GPU-contention analysis, and wrote report sections 2.3, 2.4, 5 and the pipeline parts of section 3.
Both	Wrote the requirements and the Review 1 and 2 material; ran the full test suite and the live voice tests; wrote the README, the design-trade-off and latency documents and the generative-AI disclosure; prepared the live demonstration; and reviewed the final report.

6.2 Generative-AI assistance (disclosure)

Claude (Anthropic), used through Claude Code, assisted throughout this project at the team's request, one part at a time: architecture planning, writing much of the source code and tests, debugging, writing the documentation in `docs/`, the live terminal dashboard (`terminal_viz.py`, `run_viz.py`, `viz_web.py`), the three evaluation scripts in `scripts/`, and the drafting of this report and its figures. A per-part log is in `docs/ai_usage.md`. The experiments were run on the team's own machine and every number in section 4 is produced by those scripts and saved in `docs/submission/results/`. Test sets, design decisions and conclusions were reviewed by the team.

6.3 References

1. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP-IJCNLP*.
2. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N. and Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *NeurIPS*. (Model used: sentence-transformers/all-MiniLM-L6-v2.)
3. Johnson, J., Douze, M. and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* (FAISS).
4. Radford, A. et al. (2023). Robust speech recognition via large-scale weak supervision. *ICML* (Whisper). Run through faster-whisper / CTranslate2.
5. Qwen Team (2024). Qwen2.5 technical report. arXiv:2412.15115.
6. Brown, T. et al. (2020). Language models are few-shot learners. *NeurIPS*.
7. Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*.
8. Rebedea, T. et al. (2023). NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. *EMNLP 2023, System Demonstrations*.
9. Manning, C. D., Raghavan, P. and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press (TF-IDF, cosine similarity).
10. Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson.
11. Software: Silero VAD (github.com/snakers4/silero-vad); Kokoro-82M (huggingface.co/hexgrad/Kokoro-82M); Ollama (ollama.com); LangGraph (langchain-ai.github.io/langgraph); FastAPI; PyTorch; SQLite.

Appendix A. Running the AI core and reproducing the results

Source code and test data are in the repository https://github.com/infiniteaniverse/voicechat-live; they can also be demonstrated live on the team's laptop. Requirements: Windows, an NVIDIA GPU, `ollama serve` running with `qwen2.5:3b` pulled, and the project's virtual environment.

```
# start the assistant: browser voice UI on the left, live pipeline terminal on the right
.venv\Scripts\python.exe run_viz.py            # then open http://127.0.0.1:8000/

# run the AI components' tests (real models, no mocks)
.venv\Scripts\python.exe -m pytest tests/test_memory.py tests/test_pipeline.py tests/test_latency_monitor.py
tests/test_guardrails.py

# reproduce the experiments in section 4 (each writes JSON to docs/submission/results/)
.venv\Scripts\python.exe scripts/eval_retrieval.py
.venv\Scripts\python.exe scripts/eval_extraction.py
.venv\Scripts\python.exe scripts/eval_guardrails.py

# rebuild this report's HTML from the saved results
python docs/submission/build_report.py
```

Appendix B. Live demonstration view

The dashboard animates every stage of a turn (microphone, VAD probability, transcript, guardrail verdicts, retrieved facts, token stream, sentence queue, latency bar against the 2 s budget) and is what the team uses to explain the pipeline in the live demonstration. It is a monitoring aid; the evaluated AI components are those in sections 2–4.



Figure B1. The browser page: voice UI on the left, live pipeline terminal on the right, captured after a spoken question. This was a cold first turn in a headless browser (4,055 ms), which the dashboard correctly flags as OVER budget; it is shown for the layout, not as a latency result.

Appendix C. Test material

C.1 Extraction test utterances and outputs (current prompt)

Utterance	Gold facts	Recalled	Model output
Hi, I'm Priya and I work as a pediatric nurse.	2	2	The user's name is Priya.; The user works as a pediatric nurse.
I work as a civil engineer and my name is Omar.	2	2	The user works as a civil engineer.; The user's name is Omar.
My name is Sara.	1	1	The user's name is Sara.
I'm allergic to shellfish and I live in Pune.	2	2	The user is allergic to shellfish.; The user lives in Pune.
I'm vegan, I play the violin, and my cat is named Luna.	3	3	The user is vegan.; The user plays the violin.; The user's cat is named Luna.
Please keep your answers brief from now on, I prefer them short.	1	1	The user prefers short answers.
My brother Arjun is visiting me next month.	1	1	The user's brother Arjun is visiting them next month.
I'm studying electrical engineering at university.	1	1	The user is studying electrical engineering at university.
I wake up at five every morning to go running.	1	0	NONE
My favorite programming language is Rust.	1	0	NONE
I just started a new job at a bank.	1	0	NONE
I'm a teacher and I love hiking.	2	1	The user is a teacher.

C.2 Retrieval fact store used in Experiment 1

12 facts: the user's name (Jay), job (data engineer), peanut allergy, preference for short answers, sister Maya visiting, lives in Mumbai, favourite language Rust, vegetarian, plays guitar, studies computer science (master's), dog named Bruno, wakes at six. The 26 answerable and 15 unrelated queries are listed in `scripts/eval_retrieval.py`.

C.3 Safety-check test cases

24 user messages and 10 spoken sentences, listed in `scripts/eval_guardrails.py`; per-case outcomes for both prompt styles are in `docs/submission/results/guardrails.json`.